

Part 1:

<https://studio.edgeimpulse.com/studio/1015521>

```
Sampling...
Timing: DSP 21 ms, inference 728 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.945312
  lift: 0.054687
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 21 ms, inference 716 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.929687
  lift: 0.070312
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 21 ms, inference 6 ms, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
```

Figure 1: Inference Idle state

```
lift: 0.770074
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 21 ms, inference 743 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 21 ms, inference 744 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 21 ms, inference 743 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
```

Figure 2: Inference Lift state

Part 2:

Question 1: Model architecture and ensemble flow

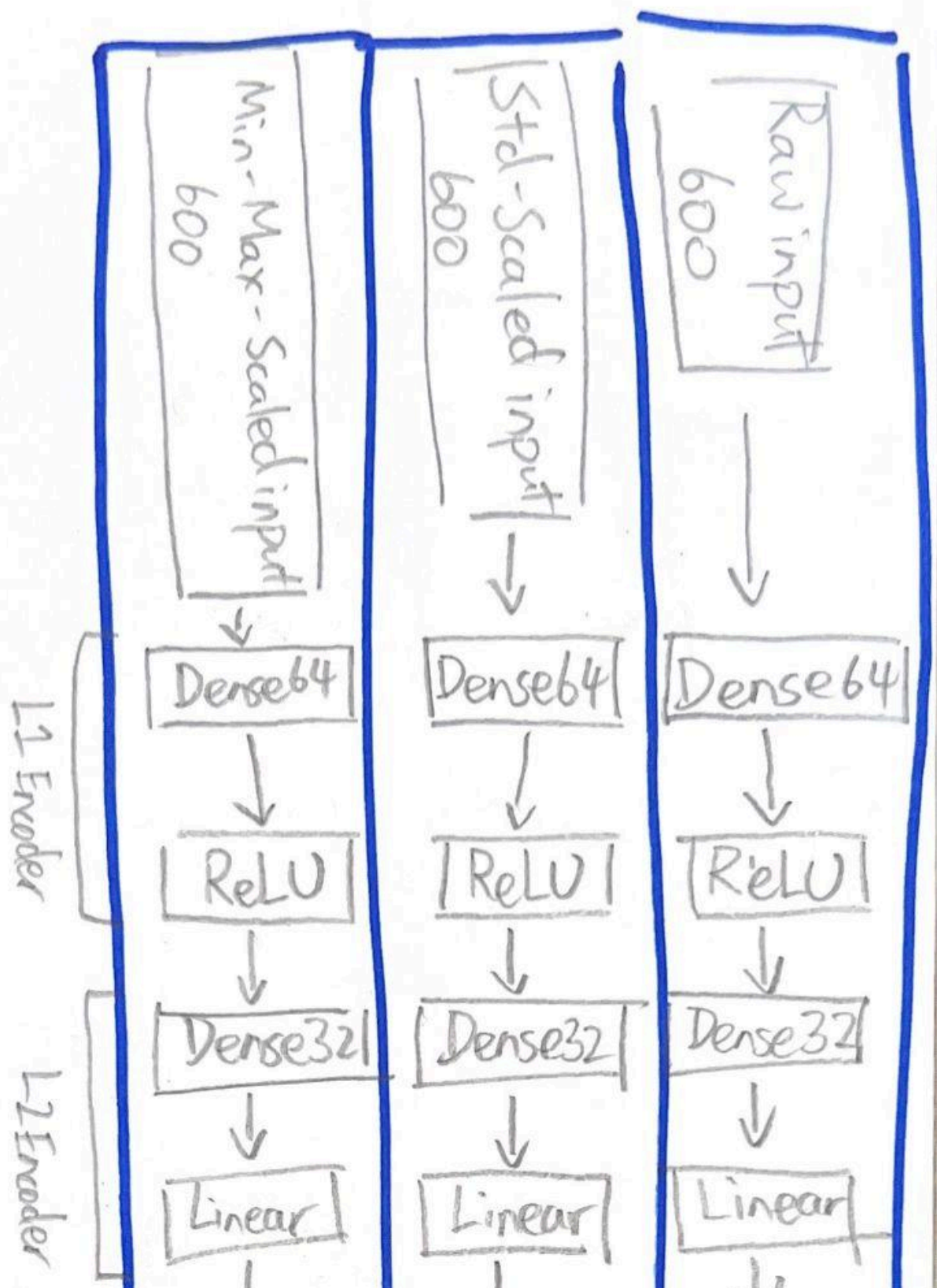
Draw a complete diagram of the full ensemble pipeline. Your diagram should include enough detail for someone to reconstruct the model architecture from your drawing.

Include the following details:

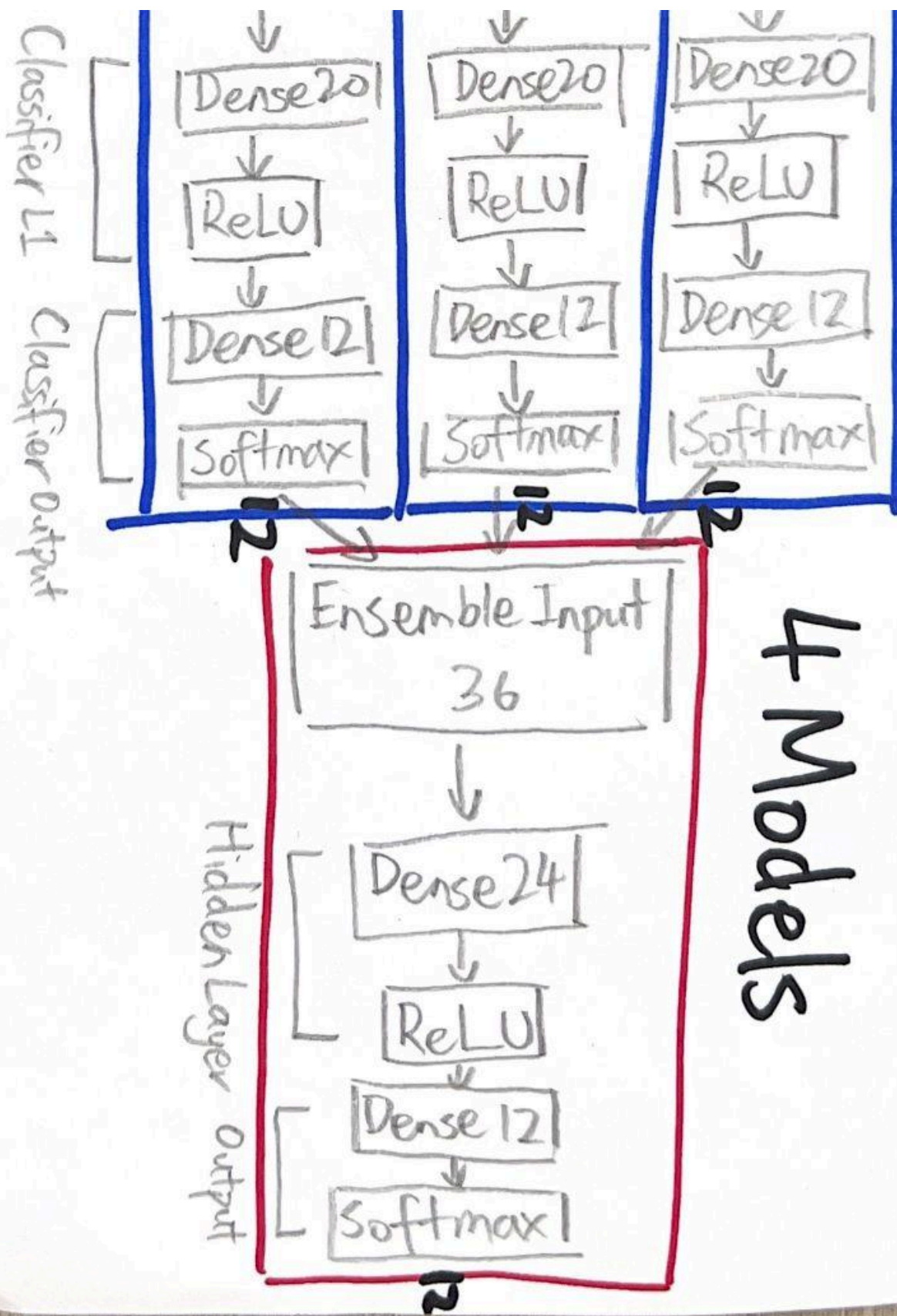
- The input window size and number of input neurons. In this notebook, each input window has 100 time steps and 6 IMU features, so the flattened input dimension is 600.
- The three input branches: raw input, standard-scaled input, and min-max-scaled input.
- The encoder architecture in each branch, including the number of hidden layers, number of neurons in each layer, and activation function used in each layer.
- The classifier architecture in each branch, including the number of input neurons, hidden-layer neurons, output neurons, and activation functions.
- The number of models used in the ensemble and how they are connected.
- The size of each branch output and how the three outputs are concatenated before being passed into the stacked meta-classifier.
- The stacked meta-classifier architecture, including input size, hidden layer size, output size, and activation functions.

Figure 3: Initial Inputs and Encoders (page 3)

Figure 4: Sub-model Classifiers and Ensemble Classifier (page 4)



4 Models



Question 2: Pruning and QAT methodology

Explain how this notebook compresses the models for TinyML deployment.

In your answer, describe:

- How pruning is applied and what sparsity schedule is used.
- Why the pruning wrappers are stripped before QAT.
- How QAT is applied after pruning.
- Why the notebook uses an additional mask-enforcement step during and after QAT instead of using only a simple pruning-then-QAT pipeline.
- How the final model is converted to an int8 TFLite model and why representative data is needed.
- How pruning and int8 quantization help when deploying to a resource-constrained device such as the Arduino Nano 33 BLE Sense.

Pruning uses a PolynomialDecay schedule that ramps sparsity from 10% to 80% over 500 steps across 5 epochs, zeroing out the lowest-magnitude weights according to that schedule. The pruning wrappers must then be stripped before QAT because they insert step-counter and mask-tracking scaffolding that is incompatible with `quantize_model`, which needs a clean Keras model to insert fake-quantization nodes. `strip_pruning` removes the scaffolding while keeping the sparse weights intact.

QAT wraps each layer with simulated int8 rounding operations and fine-tunes the model so its weights adapt to quantization noise before the final conversion. The mask-enforcement callback is necessary because gradient descent during QAT has no awareness of pruning and will immediately begin recovering the zeroed weights. The callback re-applies the binary masks every batch and epoch to hold those positions at zero.

For the final export, the TFLite converter needs a representative dataset to observe the actual range of activations flowing through each layer and set per-layer int8 scale factors correctly. Without real data, the quantization calibration would be inaccurate. Together, 80% pruning and int8 quantization reduce weight storage from 4 bytes to 1 byte per value, shrink flash usage, cut multiply-accumulate operations for sparse layers, and replace slow software float emulation with the microcontroller's native integer ALU.

Question 3: Arduino deployment and real-world behavior

Deploy the quantized model using the provided Arduino sketch for this lab. Place the Arduino Nano 33 BLE Sense flat on a stable surface, open the serial monitor, and observe the predicted activity labels.

In your answer, include:

- A screenshot of the serial monitor output.
- A short explanation of whether the output is acceptable for the resting board condition.
- Any issues you observe, such as unstable predictions, unexpected labels, sensor noise, scaling mismatch, or mismatch between the training data and the real board orientation.

- One or two practical changes that could improve deployment reliability.

```

Meta-classifier scores: 0.0039, 0.5273, 0.0625, 0.0039, 0.0039, 0.0117, 0.3750, 0.0000, 0.0039, 0.0039, 0.0039, 0.0000
-----
Final prediction: Sitting and relaxing
Class index: 1
Confidence score: 0.5273
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0156, 0.9805, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Standard-scaled model scores: 0.0000, 0.0117, 0.0117, 0.0000, 0.0000, 0.0000, 0.9609, 0.0117, 0.0000, 0.0039, 0.0000, 0.0000
Min-max-scaled model scores: 0.0000, 0.0000, 0.4414, 0.0156, 0.4023, 0.0078, 0.0156, 0.1055, 0.0039, 0.0000, 0.0000, 0.0117
Meta-classifier scores: 0.0039, 0.5273, 0.0625, 0.0039, 0.0039, 0.0117, 0.3750, 0.0000, 0.0039, 0.0039, 0.0039, 0.0000
-----
Final prediction: Sitting and relaxing
Class index: 1
Confidence score: 0.5273
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0156, 0.9805, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Standard-scaled model scores: 0.0000, 0.0117, 0.0117, 0.0000, 0.0000, 0.0000, 0.9648, 0.0117, 0.0000, 0.0039, 0.0000, 0.0000
Min-max-scaled model scores: 0.0000, 0.0000, 0.4414, 0.0156, 0.4023, 0.0078, 0.0156, 0.1055, 0.0039, 0.0000, 0.0000, 0.0117
Meta-classifier scores: 0.0039, 0.5273, 0.0625, 0.0039, 0.0039, 0.0117, 0.3750, 0.0000, 0.0039, 0.0039, 0.0039, 0.0000
-----
Final prediction: Sitting and relaxing
Class index: 1
Confidence score: 0.5273
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75

```

Figure 5: Deployment on a stationary table

The serial monitor shows the model consistently predicting "Sitting and relaxing" (class index 1) with a confidence score of 0.5391 across every inference window, with identical score vectors repeating each time. For a board resting flat on a desk, a stationary low-movement class is a reasonable prediction, so the label itself is not unreasonable.

However, the complete lack of variation across windows is a sign of something deeper than just a stable board. The raw and standard-scaled branches both concentrate nearly all probability mass on class 6 (frontal elevation of arms, score ~0.98 and ~0.95 respectively), while the min-max-scaled branch spreads probability more diffusely across classes 2, 4, and 7. The meta-classifier then resolves this disagreement toward class 1 at 0.54, but the individual branch outputs suggest a distribution mismatch rather than genuine recognition of a sitting posture.

The most likely cause is that the board's static gravity vector does not match the orientation or unit convention of the mHealth training data, causing all three branches to receive out-of-distribution inputs and lock into fixed latent-space regions. Two practical fixes would be to apply a temporal majority-vote filter over several consecutive windows to smooth any instability, and to verify that the accelerometer output units and axis orientation in the Arduino sketch exactly match the mHealth dataset's recording conventions before passing values to the scalars.

Question 4: Deployment Behavior on the Arduino

After deploying the ensemble model on the Arduino Nano 33 BLE Sense / Sense Rev2, did you observe any unexpected prediction behavior? For example, did the model repeatedly predict the same class even when the board was moved differently?

Briefly explain why this may happen. In your answer, discuss possible causes such as differences between the original training dataset and the live Arduino sensor data, sensor placement, axis orientation, unit conversion, feature scaling, or mismatch between the motion patterns used during training and testing.

The deployment results show two distinct behavioral regimes. When the board is stationary, the model locks onto "Sitting and relaxing" every cycle with near-identical score vectors, driven by the static 1g gravity signature on one accelerometer axis that has no close match in the mHealth training distribution. When the board is moved with larger physical motions, the prediction shifts to high-movement classes like "Running" and "Lying down," confirming the ensemble is sensitive to motion magnitude, but the low confidence scores (0.43-0.48) and inconsistent branch votes reveal that the live motion signatures still do not closely resemble any specific training class.

The core issue is the gap between the original data collection setup and the live Arduino sensor. The mHealth dataset was recorded from sensors worn on a human body during structured activities, capturing correlated, biomechanically coherent motion across all six IMU channels simultaneously. The Arduino being waved by hand on a desk produces motion that superficially resembles high-activity classes in terms of magnitude, but lacks the specific frequency content, axis correlations, and postural context present in the training data. The result is unstable, low-confidence predictions that shift between high-movement classes rather than tracking any real activity. Retraining on data collected directly from the Arduino in the target deployment orientation would be the most reliable path to stable, meaningful predictions.